



TRAMA

Ingeniería de Software

La estructura detrás del servicio.

QUIÉNES SOMOS

Especialistas en formalización digital

Firma de ingeniería de software que convierte operaciones de servicio informales —las que hoy viven en WhatsApp, llamadas y contactos— en plataformas trazables, auditables y preparadas para crecer.

PROPUESTA DE VALOR

- Trazabilidad completa: cada decisión técnica responde a una necesidad documentada del cliente.
- Arquitectura preparada para operar múltiples empresas sin rediseño.
- Integramos lo que ya existe; construimos solo lo que diferencia al negocio.



Trama

Estructura que sostiene un tejido. Es lo que le falta a una red de aliados que funciona por contactos.

Misión y visión



MISIÓN

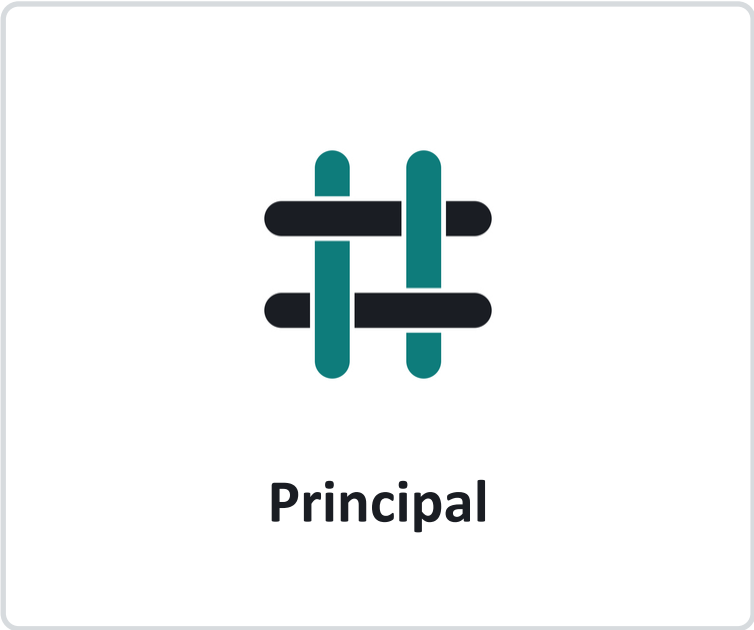
Diseñar y construir plataformas de software que permitan a empresas de servicios formalizar su operación, con arquitectura documentada, decisiones justificadas y calidad verificable.



VISIÓN

Ser reconocidos en 2030 como la firma de referencia en Colombia para la formalización digital de operaciones de servicio distribuidas.

Uso del logo - Color - tipografía



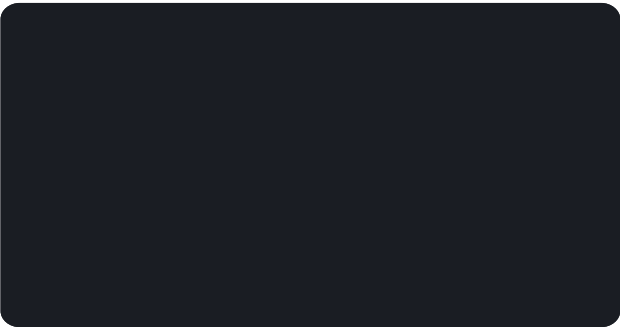
Fondos claros



Impresión y documentos



Fondos oscuros



#1A1D23

Texto y fondos oscuros



#0E7C7B

Acento, títulos, énfasis



#C77800

Alertas y riesgos. Uso escaso



#5A6068

Texto secundario y notas



#FFFFFF

Fondo por defecto

Tipografía

Una sola familia sans en todo el proyecto.

Título

30 pt · Semibold

Subtítulo

15 pt · Semibold

Cuerpo

12 pt · Regular

Nota

10 pt · Regular

Servicios

01

Ingeniería de requisitos

Identificación, análisis y documentación de necesidades para convertirlas en requisitos claros, verificables y trazables que orienten el desarrollo de la solución.

02

Diseño de software como solución

Diseño de soluciones de software definiendo arquitectura, componentes, atributos de calidad y decisiones técnicas que garanticen escalabilidad, seguridad y mantenibilidad.

03

Desarrollo de proyectos de software

Construcción de soluciones de software a la medida, desde la planificación y desarrollo hasta las pruebas e implementación, alineadas con los objetivos y procesos del negocio.

04

Mantenimiento a nuestro software

Evolución, optimización y soporte de soluciones existentes mediante corrección de errores, mejoras funcionales, actualizaciones técnicas y adaptación a nuevas necesidades.

LO QUE NOS HACE DIFERENTES

Diseño integral de la solución

Diseñamos cada solución de manera integral, considerando arquitectura, tecnología, seguridad, calidad y operación durante todo su ciclo de vida.

01 Arquitectura y escalabilidad

Soluciones modulares y escalables, preparadas para crecer y evolucionar con el negocio.

02 Selección tecnológica

Evaluamos tecnologías, frameworks y patrones según requerimientos, restricciones y proyección.

03 Integración de sistemas

Arquitecturas de integración seguras con ERP, CRM, pasarelas de pago y APIs de terceros.

04 Seguridad desde el diseño

Control de acceso, protección de datos y trazabilidad definidos desde la arquitectura.

05 QA y automatización

Estrategias de prueba, validación por escenarios y automatización para asegurar confiabilidad.

06 DevOps y ciclo de vida

Versionamiento, CI/CD y gestión de DEV, QA y PROD para una evolución controlada.

Estructura de la empresa

Estructura permanente, independiente de cualquier proyecto.



Equipo del proyecto

Los representantes de cada area para este proyecto son:

Gestión de Proyecto

SM



PO



Infraestructura

DevOps



Calidad

QA



Ingeniería



Frontend



Frontend



Backend

Equipo del proyecto

Comité de Arquitectura

ARQUITECTA



ARQUITECTO



ARQUITECTO



ARQUITECTO



ARQUITECTO



ARQUITECTA



ARQUITECTO

Mesa de Arquitectura

Ceremonia con periodicidad según se requiere donde se toman y registran todas las decisiones arquitectónicas.

1

Preparación

- Cada dominio tiene un responsable
- Ficha enviada 24 h antes
- Mínimo dos alternativas reales

2

Deliberación

- Quórum de 5 de 7
- 20 minutos por decisión
- Un integrante argumenta en contra

3

Registro

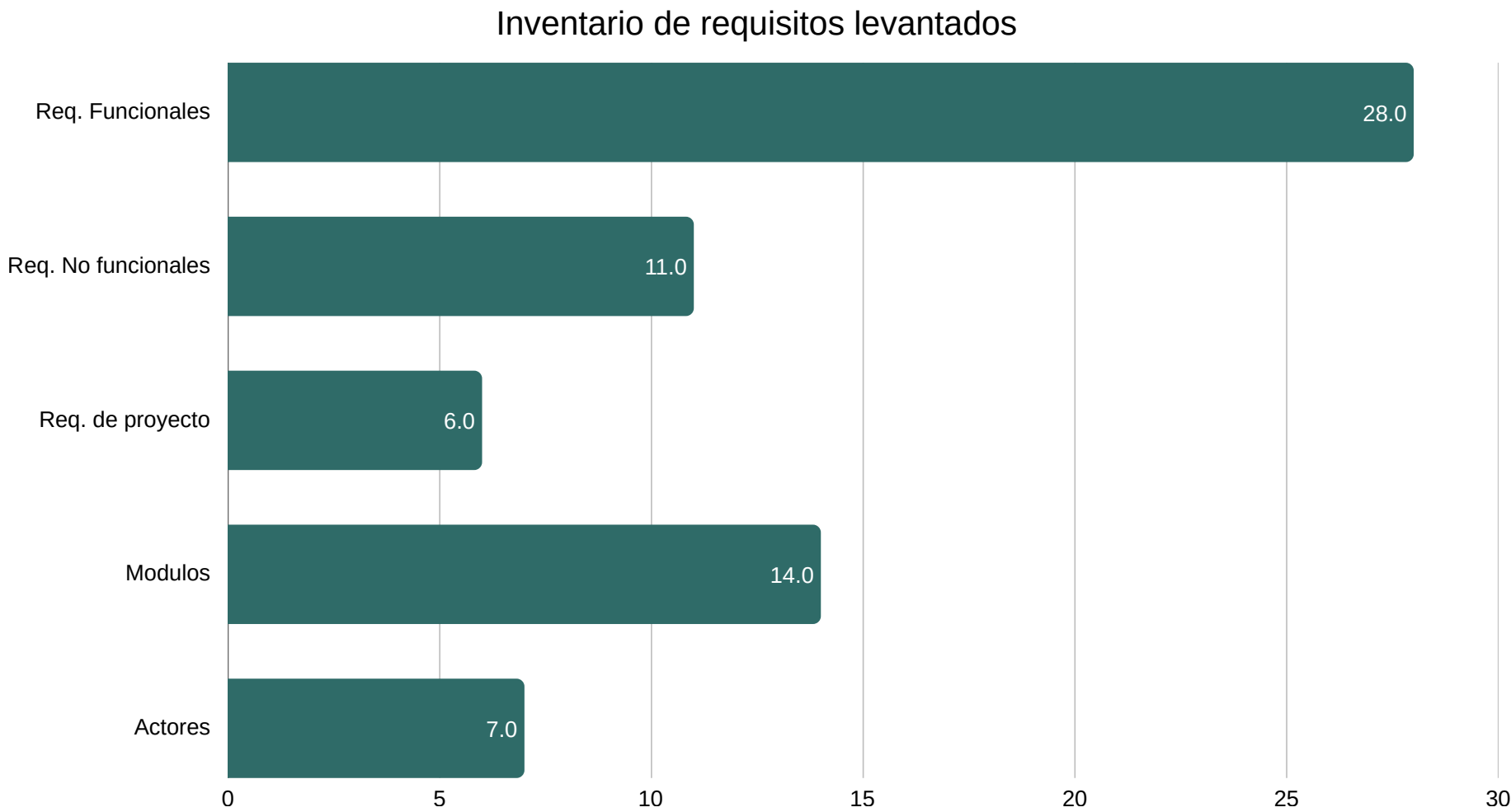
- ADR numerado por decisión
- Redactor distinto al proponente
- El disenso queda documentado

- Los roles de facilitador, redactor y contradictor rotan en cada sesión.
- Ninguna decisión se aprueba sin alternativas ni sin el requisito que la justifica.

01

Ingeniería de requisitos

Convertimos la operación informal del cliente en requisitos clasificados, priorizados y tecnológicamente neutrales.



- 1

Del problema al requisito
Separamos síntomas, causas y necesidades antes de escribir funcionalidades.
- 2

Neutralidad tecnológica
Ningún requisito nombra stack: la decisión técnica llega después y se justifica.
- 3

Alcance delimitado
MVP frente a segundo incremento, con lo excluido declarado explícitamente.
- 4

Vacíos declarados
Lo que no sabemos se registra como vacío, no se rellena con supuestos.

Diseño de software como solución

OBJETIVO GENERAL

Formalizar digitalmente la operación de servicios del cliente mediante una plataforma SaaS multi-tenant que soporte el ciclo solicitud–cotización–ejecución–calificación–cierre, con trazabilidad operativa y configuración propia por empresa.



CICLO DEL SERVICIO QUE HABILITA EL PRODUCTO



Solicitud



Cotización



Ejecución



Calificación



Cierre

Alcance del producto

MVP (este corte) · EP-01 a EP-06
Plataforma multi-tenant, directorio de actores, catálogo y cobertura, ciclo del servicio, comunicación y tarifario.

2º incremento · EP-07 y EP-08
Pagos y facturación, quejas, comercialización y administración avanzada.

Limitaciones del producto

Cobertura por zonas, no por radio ni geolocalización en tiempo real.
Fuera del MVP: pasarela de pago, facturación electrónica y consola de comercialización.

Pagos centralizados vía operador certificado; PCI DSS recae en el operador.
Aislamiento estricto entre tenants sin despliegue específico por empresa.

Product Backlog

Épicas, historias, spikes y candidatas a Sprint 1

8

Épicas
(6 MVP · 2 en 2º incremento)

40

Historias de usuario

13

Spikes técnicos
(Sprint 0)

4

Bugs previsibles identificados

Mapa de épicas

EP-01	Plataforma multi-tenant	MVP
EP-02	Directorio de actores	MVP
EP-03	Catálogo y cobertura	MVP
EP-04	Ciclo del servicio	MVP
EP-05	Comunicación	MVP
EP-06	Tarifario	MVP
EP-07	Pagos y facturación	2º incremento
EP-08	Operación y comercialización	2º incremento

Sprint 0 — actual

- 13 spikes técnicos de investigación (uno condicional)
- 2 críticos bloquean Sprint 1: SP-01.1.1 (aislamiento multi-tenant) y SP-01.2.1 (proveedor de identidad)
- No se compromete código de producto en este corte

Sprint 1 — candidata (no iniciado)

5 historias · 21 pts · fundación multi-tenant + identidad

- US-01.1.1 Registrar tenant — Crítica (5)
- US-01.1.2 Configurar documentos por tipo de aliado (5)
- US-01.2.1 Login restringido al tenant — Crítica (5)
- US-01.2.2 Recuperar contraseña (3)
- US-01.2.3 Asignar roles a usuarios (3)

03

Formulación y Desarrollo de proyectos de software



OBJETIVO GENERAL

Desarrollar la aplicación “**MANI**”, plataforma SaaS multi-tenant que soporte el ciclo solicitud–cotización–ejecución–calificación–cierre, con trazabilidad operativa y configuración propia por empresa.

OBJETIVOS ESPECIFICOS

- Implementar una plataforma multi-tenant con aislamiento estricto de datos, configuración y usuarios entre empresas (RF-01 a RF-04).
- Construir el directorio diferenciado de aliados (persona natural, empresa, empleado directo) y clientes (persona natural, empresa), con verificación configurable por tenant (RF-05 a RF-09).
- Habilitar el catálogo de categorías de servicio y la declaración de cobertura por zonas (RF-10, RF-11).
- Cubrir el ciclo del servicio de extremo a extremo: solicitud, cotización, ejecución, calificación y cierre, sin dobles asignaciones (RF-12 a RF-19).
- Proveer mensajería y notificaciones asociadas a cada servicio (RF-20, RF-21).
- Mantener un tarifario de referencia por categoría con alerta cuando una cotización se sale de rango (RF-22, RF-23).

Módulos funcionales

Seis módulos componen el MVP y dos quedan para el segundo incremento. Cada módulo corresponde a una épica del Product Backlog.

MVP · ESTE CORTE

M-01

Plataforma multi-tenant

EP-01

Aislamiento de datos y configuración por empresa

M-02/03

Directorio de actores

EP-02

Aliados (natural, empresa, empleado) y clientes

M-04

Catálogo y cobertura

EP-03

Categorías por tenant y cobertura declarada por zonas

M-05..08

Ciclo del servicio

EP-04

Solicitud, cotización, ejecución y calificación

M-09

Comunicación

EP-05

Mensajería por servicio y notificaciones

M-11

Tarifario

EP-06

Tarifas de referencia y alerta fuera de rango

SEGUNDO INCREMENTO · FUERA DE ESTE CORTE

M-10

Pagos y facturación

EP-07

Cobro al cliente y liquidación al aliado

M-12..14

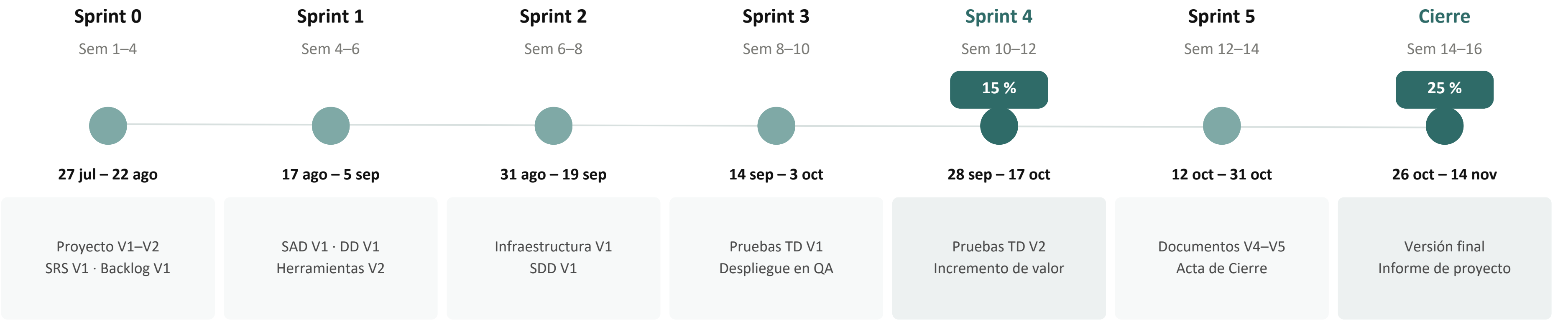
Quejas y administración

EP-08

Quejas, comercialización y métricas por tenant

Cronograma

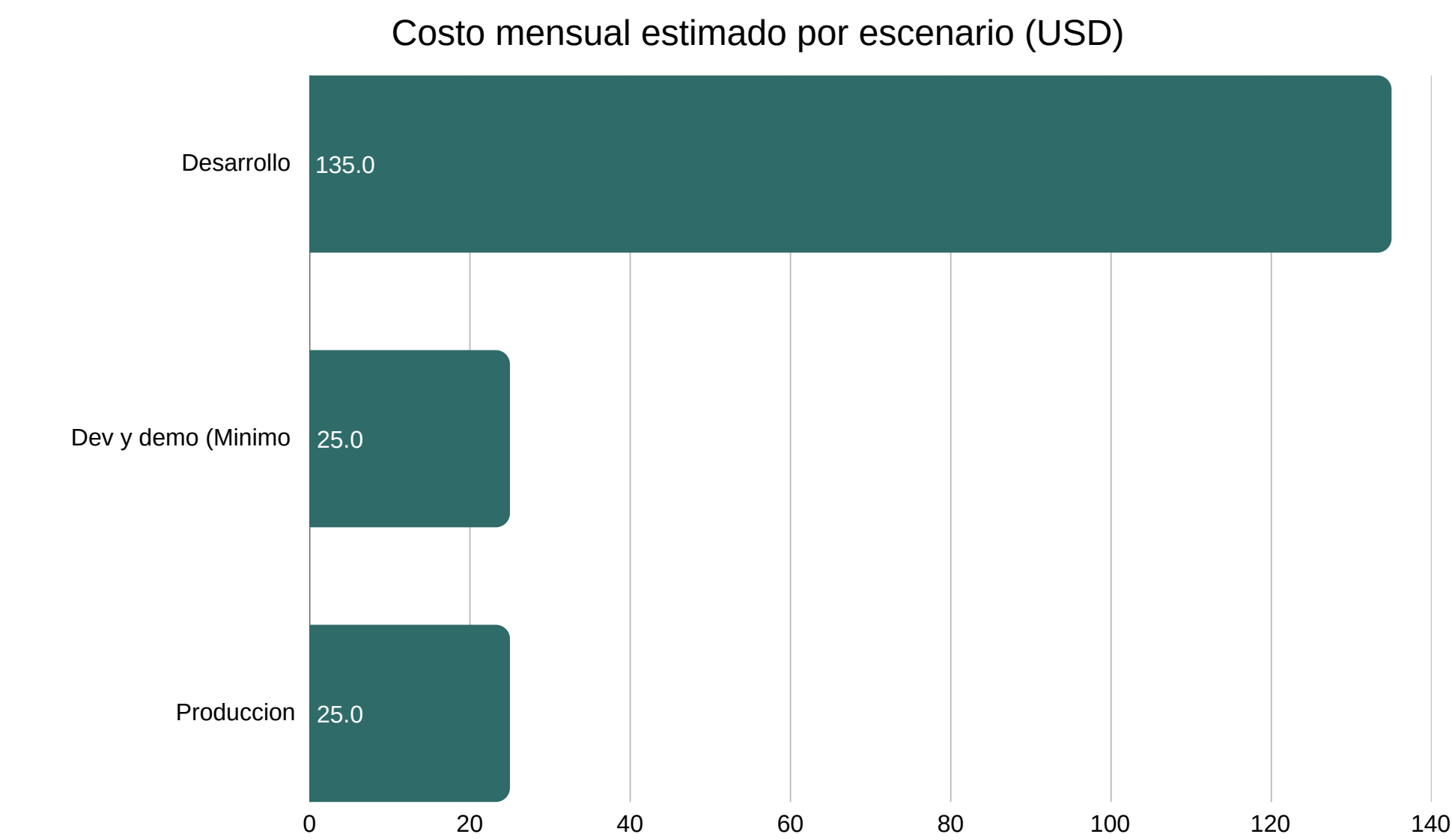
Anclado al Calendario General 2026 (Javeriana, segundo período). Inicio 27 de julio; sprints de dos semanas académicas; Review en semanas 6, 8, 10, 12 y 14.



Peso total del proyecto: 40 % (15 % Sprint 4 + 25 % Cierre); las demás entregas son formativas. Los festivos del 17 de agosto, 12 de octubre y 2 de noviembre restan una sesión, y la Semana Javeriana (28-30 sep) puede correr el tramo final una semana.

Presupuesto

Cálculo de referencia con precios públicos vigentes (agosto 2026). El presupuesto de producto depende de dos decisiones que aún no tienen ADR.



Herramientas con costo

Microsoft 365 / OneDrive — \$42/mes
Postman Team — \$57/mes
Figma Professional — \$36/mes
Jira, GitHub, Discord, Miro y k6 operan en tier gratuito.

Lo que mantiene abierto el rango, el Stack

Persistencia — MongoDB Atlas ($\approx$ \$171/mes) frente a Supabase Pro (\$25/mes); excluyentes hasta el spike SP-01.1.1.
Orquestación — Kubernetes suma $\approx$ \$450–650/mes y aún no tiene driver de calidad que lo justifique.

Total del proyecto durante las 16 semanas: $\approx$ \$500 USD. No se fija presupuesto de producto en firme hasta elección de Stack.

Entregables

Ocho líneas documentales que evolucionan por versiones sprint a sprint, más el producto desplegado y los artefactos de cierre.

	Sprint 0	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Cierre
SRS	V1	V2	V3	V4	V5	V5	Final
Product Backlog	V1	V2	V3	V4	V5	V5	Final
Herramientas y lineamientos	V1	V2	V3	V3	V3	V3	Final
SAD		V1	V2	V3	V4	V5	Final
DD · datos y contratos		V1	V2	V3	V4	V5	Final
Infraestructura			V1	V2	V3	V4	Final
Diseño SDD			V1	V2	V3	V4	Final
Pruebas TD				V1	V2	V3	Final

Producto

Despliegue en QA e incremento de valor desde el Sprint 3.

Pruebas

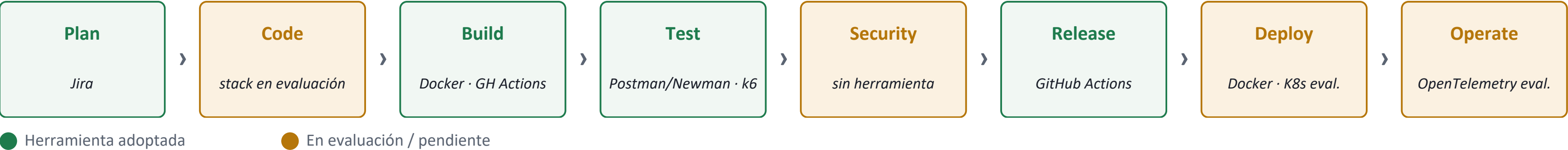
Informe de pruebas en cada review a partir del Sprint 3.

Cierre

Acta de Cierre, resultados, lecciones aprendidas y conclusiones.

Políticas de DevOps

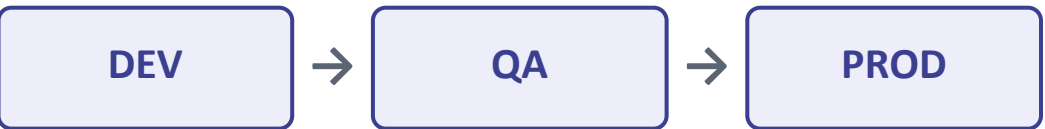
Flujo DevSecOps, ramas, ambientes y decisiones pendientes — Gestión de Desarrollo



Ramas & Pull Requests

- main (release) · develop (integración)
- feature/US-xx-..., fix/..., spike/SP-xx-...
- Todo cambio entra por PR: CI en verde + 1 revisión de otro integrante
- Fallo de CI en develop: se atiende en < 4 h
- Liberación a main: la decide DevOps titular

Ambientes & contenedores



- Contenerización DEV/QA con Docker y Docker Compose
- Ningún despliegue a PROD sin pasar por DEV y QA con CI en verde
- Orquestación (Kubernetes): en evaluación por la Mesa de Arquitectura

Pendiente de decisión

- Stack de desarrollo (Code) — backend/frontend
- Herramienta de Security Testing: SAST, análisis de dependencias y gestión de secretos
- Kubernetes — orquestación y autoescalado
- Nomenclatura exacta de ramas (confirmar antes de Sprint 1)
- Observabilidad: OpenTelemetry (evaluar)

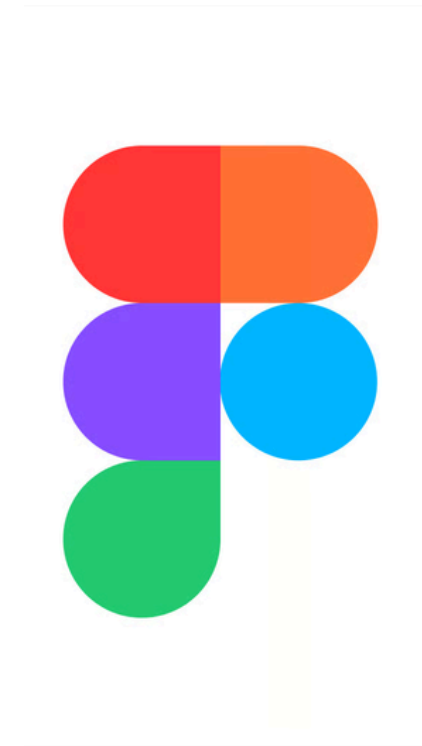
Herramientas por rol y fase

Fase DevSecOps	Rol	Herramientas
Requirements / Mockups	Product Owner	Miro, Figma, Mermaid, Jira, Confluence
Plan	Scrum Master	GitHub Projects, GitLab Issues, Wiki
Code / Build	Backend	Dart, Supabase
Code / Build	Frontend	Flutter, Dart, Figma
Test / Security Testing	QA	GitHub Issues, Postman, Newman, Dashboard Supabase, k6, Emulador
Release / Deploy / Operate / Monitor	DevOps	GitHub Actions, Docker, Docker Compose, OpenTelemetry

Herramientas - ¿por qué cada una?

Product Owner (Requirements/Mockups)

- Miro: permite mapear el flujo del usuario y hacer lluvia de ideas colaborativa en tiempo real con todo el equipo
- Figma: para bocetar cómo se verá la app antes de programar, y que el equipo valide el diseño
- Mermaid: genera diagramas (flujos, arquitectura) directo desde texto/código, fácil de versionar junto al proyecto
- Jira: gestión de historias de usuario y backlog con seguimiento de estado
- Confluence: documentación centralizada de PRDs y ADRs, accesible para todo el equipo



miro

Herramientas - ¿por qué cada una?

Scrum Master (Plan)



- GitHub Projects / GitLab Issues: Permite planificar y dar seguimiento a tareas directamente conectado al código, sin herramientas externas.
- Wiki: documentación rápida y accesible del proyecto para todo el equipo.

Herramientas - ¿por qué cada una?

Backend (Code/Build)



Dart

■ BaaS —SupraBase: Usamos Supabase porque ofrece un backend rápido, seguro y escalable con base de datos, autenticación y almacenamiento integrados.

Herramientas - ¿por qué cada una?

Frontend (Code/Build)

- Flutter: un solo código para Android e iOS, reduce tiempo de desarrollo con equipo pequeño y plazo corto (4 meses)
- Dart: lenguaje con tipado fuerte y null safety, reduce errores en tiempo de ejecución
- Figma: permite diseñar y prototipar la interfaz antes de programar, evitando reprocesos de código por cambios de diseño



Flutter



Dart

Herramientas - ¿por qué cada una?

QA (Test/Security Testing)

- Postman / Newman: prueba que las APIs respondan correctamente (precios, cotizaciones, pagos) y automatiza esas pruebas dentro del pipeline CI/CD
- k6: simula muchos usuarios al mismo tiempo para probar que el sistema aguante alta concurrencia (clave en el motor de matching)
- GitHub Issues: conecta los bugs encontrados directo con el repo de código
- Dashboard Supabase / Google Sheets: verifican que los datos se guarden correctamente y con integridad
- Emulador: simula el comportamiento real de la app en distintos dispositivos antes de publicarla



Herramientas - ¿por qué cada una?

DevOps

(Release/Deploy/Operate/Monitor)

- GitHub Actions: automatiza el proceso de integración y despliegue continuo (CI/CD) cada vez que se sube código nuevo
- Docker / Docker Compose: empaqueta la aplicación con todas sus dependencias, garantizando que funcione igual en cualquier ambiente
- Grafana: Sirve para visualizar, analizar y monitorizar datos y métricas en tiempo real



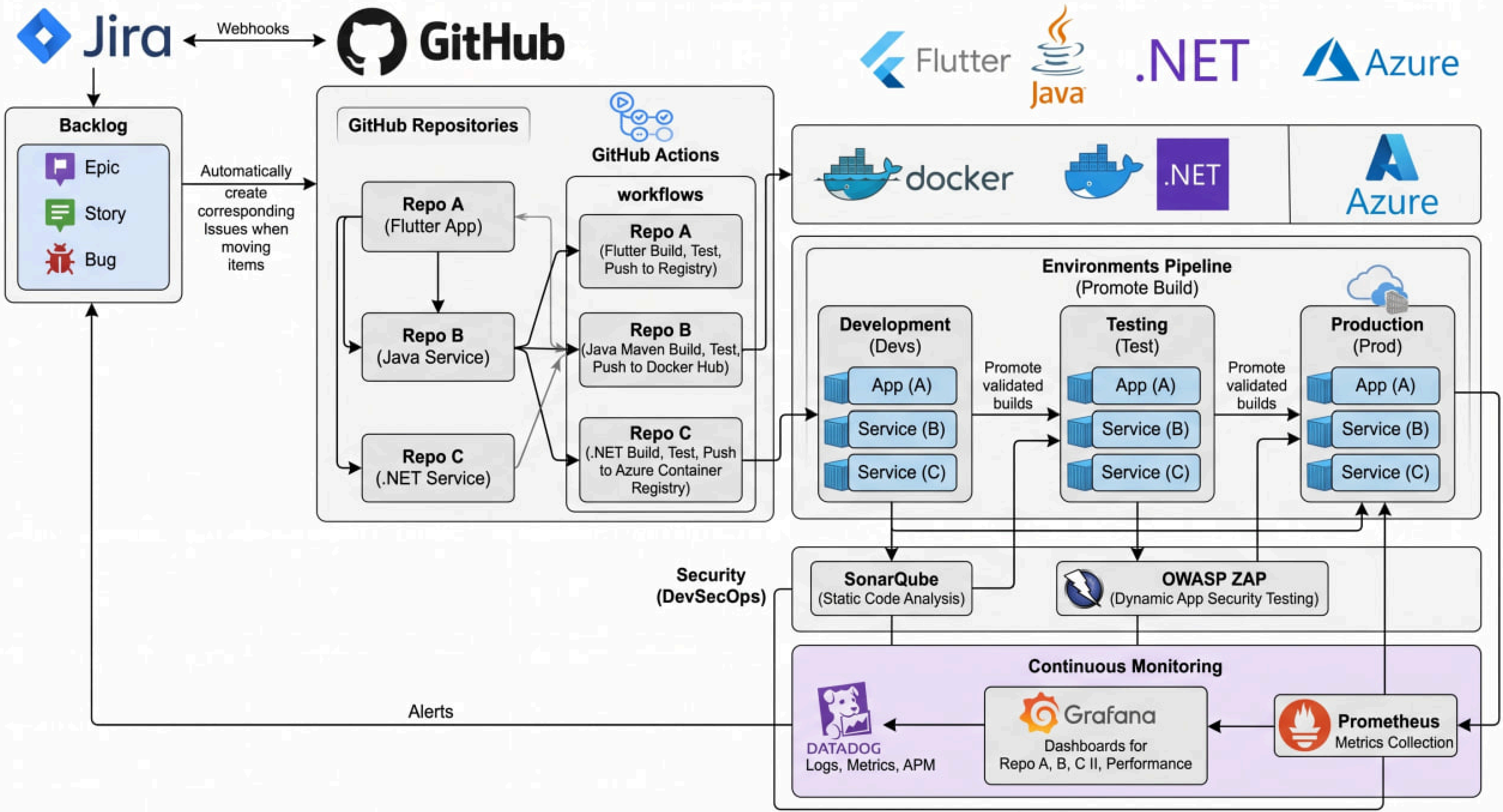
GitHub Actions



Grafana



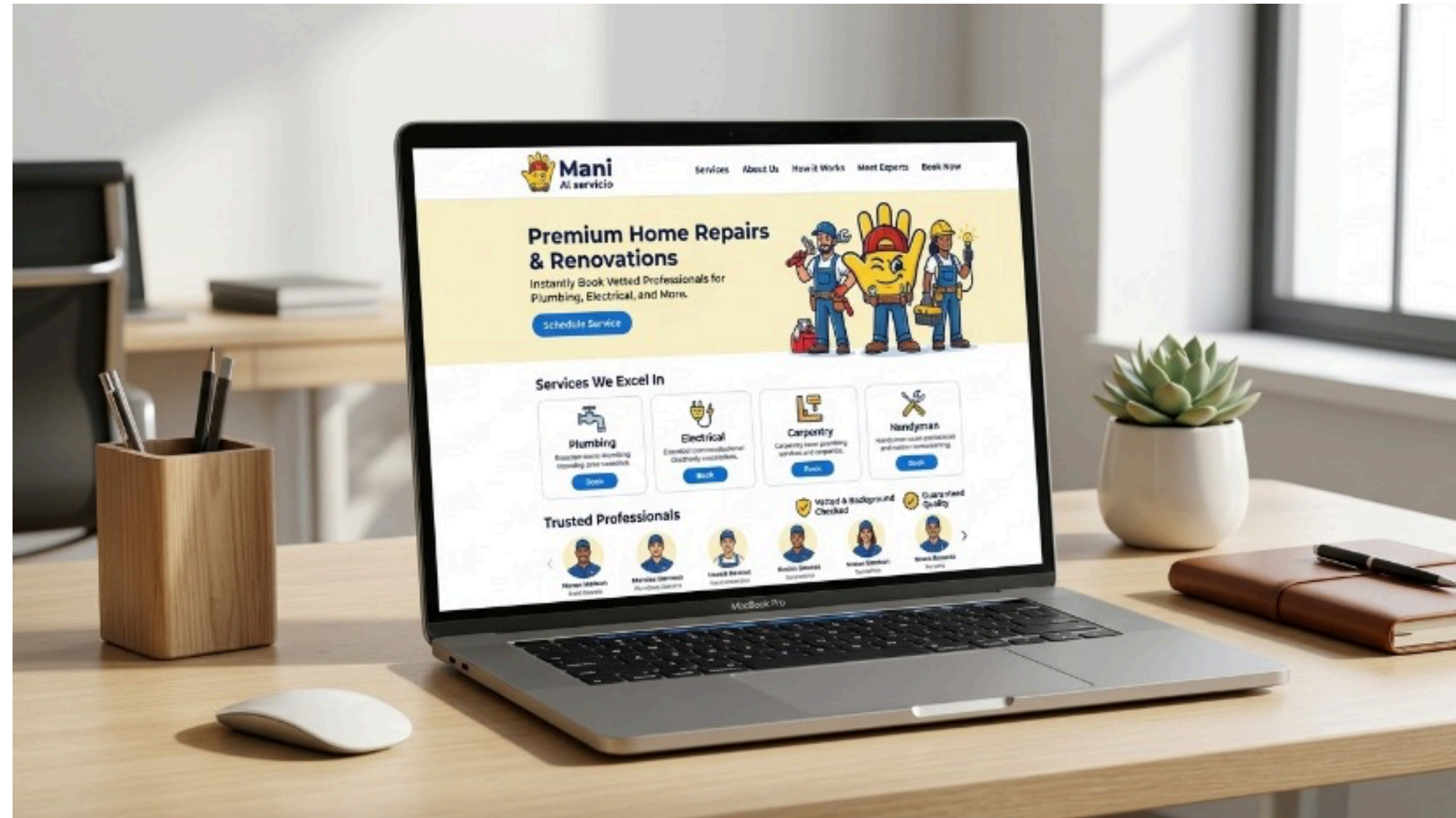
docker®





04

Mantenimiento de Producto



Mantenimiento de producto

Diseñamos y construimos el producto, pero el valor se sostiene después: el mantenimiento define dónde vive la solución, cómo se corrige, cómo evoluciona y cómo se conserva la trazabilidad de sus decisiones.

DÓNDE VIVE EL PRODUCTO





TRAMA

La estructura detrás del servicio.

Esta identidad aplica a todos los documentos y presentaciones presentados por TRAMA.